

Task : Improper Session Management

Name: Anjelina Behera

Course : CyberSecurity

Date: 5-08-26

Objective -

The objective of this assignment was to analyze the session management mechanism of a web application, identify a weakness in how user privileges were handled, and demonstrate how improper session management can lead to privilege escalation. The assignment also involved using Burp Suite to inspect HTTP requests and cookies, identifying sensitive information exposed within the application.

Introduction -

This assignment focused on identifying weaknesses in session handling, examining HTTP requests using Burp Suite, and understanding why authorization decisions must always be enforced on the server side.

Tools Used :

- Kali Linux – penetration Testing operating system
- Burp Suite Community Edition – HTTP proxy and request inspection
- Web Browser – Accessing the target web application
- Browser Developer Tools - Viewing page source and HTML
- Tryhackme target machine – Vulnerable web application

Prerequisites :

What is Session Management ?

When a user logs into a website, the server creates a session to remember that the user has been authenticated. Instead of sending the username and password with every request, the browser sends a session identifier (usually stored as a cookie). The server uses this identifier to associate requests with the correct authenticated user.

A secure session management implementation ensures that:

- Session identifiers are unpredictable.
- Session data cannot be modified by the client.
- User roles and permissions are validated on the server.
- Sessions expire appropriately after inactivity or logout.

What are Cookies ?

Cookies are small pieces of data stored by the browser and sent with future HTTP requests. Common uses include:

- Session management
- User preferences
- Authentication
- Personalization

What is Privilege Escalation ?

Privilege escalation occurs when a user gains permissions beyond those originally granted. In secure applications, privilege checks must always be performed on the server before granting access to sensitive functionality.

Methodology :

Step 1- Inspecting the Client-Side Source Code

The first step involved examining the HTML source code of the login page using the browser's View Page Source option. The source code contained credentials for the exercise:

- username : john
- password : babayaga

Security observation

Embedding credentials within client-side source code is insecure because any user can inspect the page source and retrieve them. Sensitive information should never be exposed in HTML, JavaScript, or other client-delivered content.

Step 2- Authenticating to the Application

I used the credentials to log into the web application. After authentication , I was redirected to :

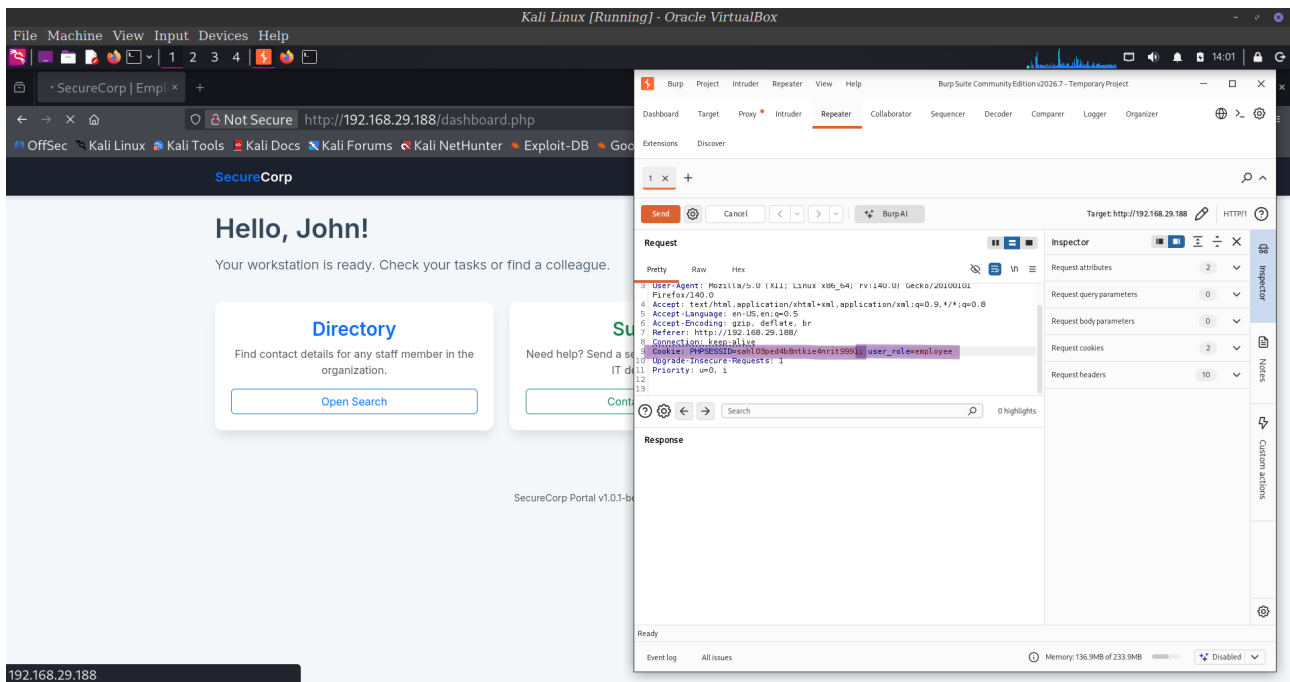
`/dashboard.php`

This showed that the login was successful and an authenticated session had been established.

Step 3- Intercepting HTTP requests with Burp Suite

Burp Suite was configured as the browser's proxy to observe the communication between the browser and the web application. HTTP requests were intercepted and examined to understand how the application maintained the authenticated session.

The request headers included cookie information :



Step 4- Analyzing the Session Cookie

The intercepted cookie contained a value indicating the user's role:

`user_role=employee`

This suggested that the application was storing authorization information directly in a client-controlled cookie.

Security Observation

This represents an insecure design because cookies stored on the client can potentially be modified.

Authorization decisions should be based on server-side session data rather than trusting client-supplied role information.

Step 5- Accessing the Admin role

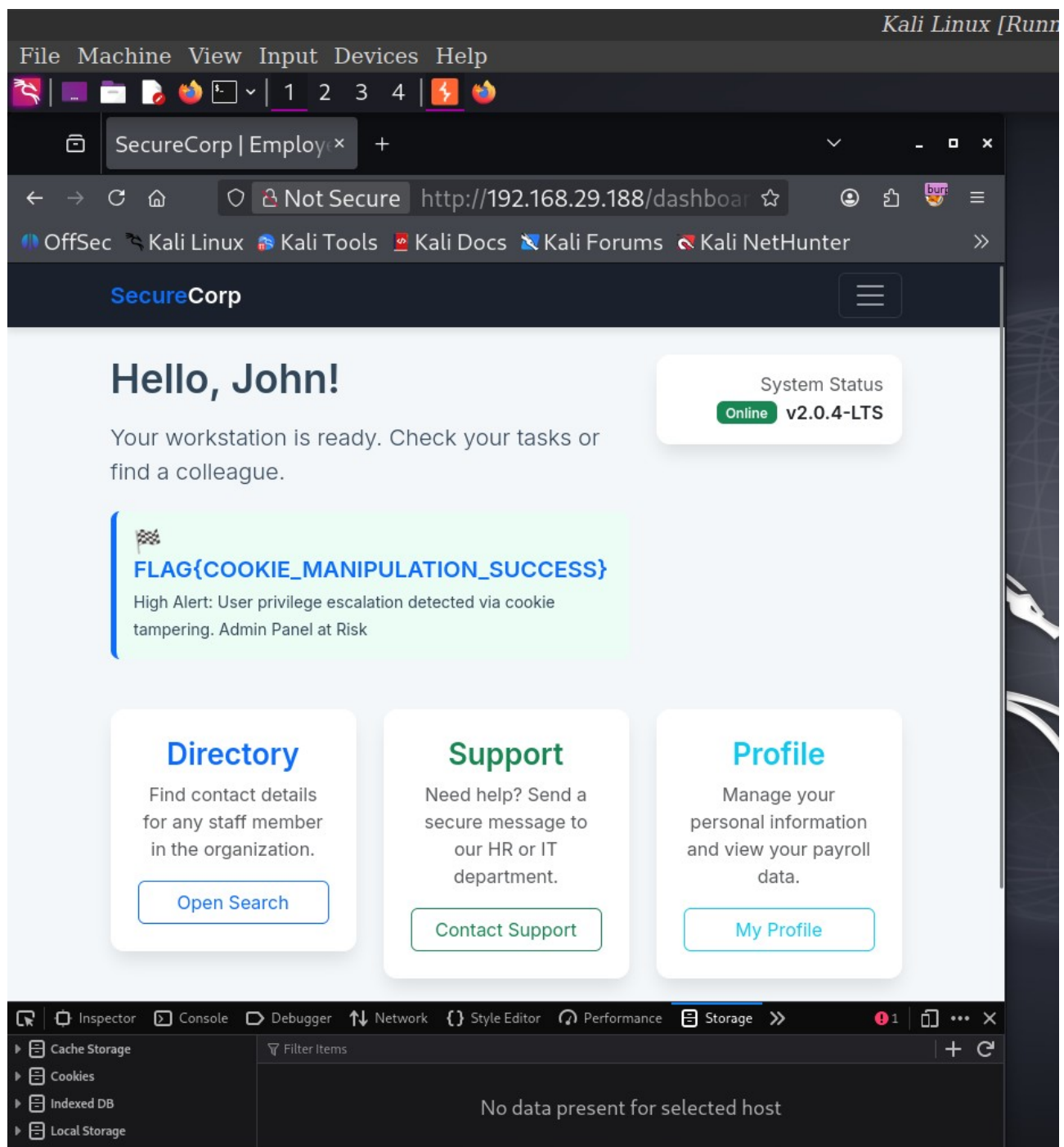
I sent the request to the repeater in Burp Suite and changed the `user_role` from `employee` to `admin`, the application then allowed access to the administrator dashboard.

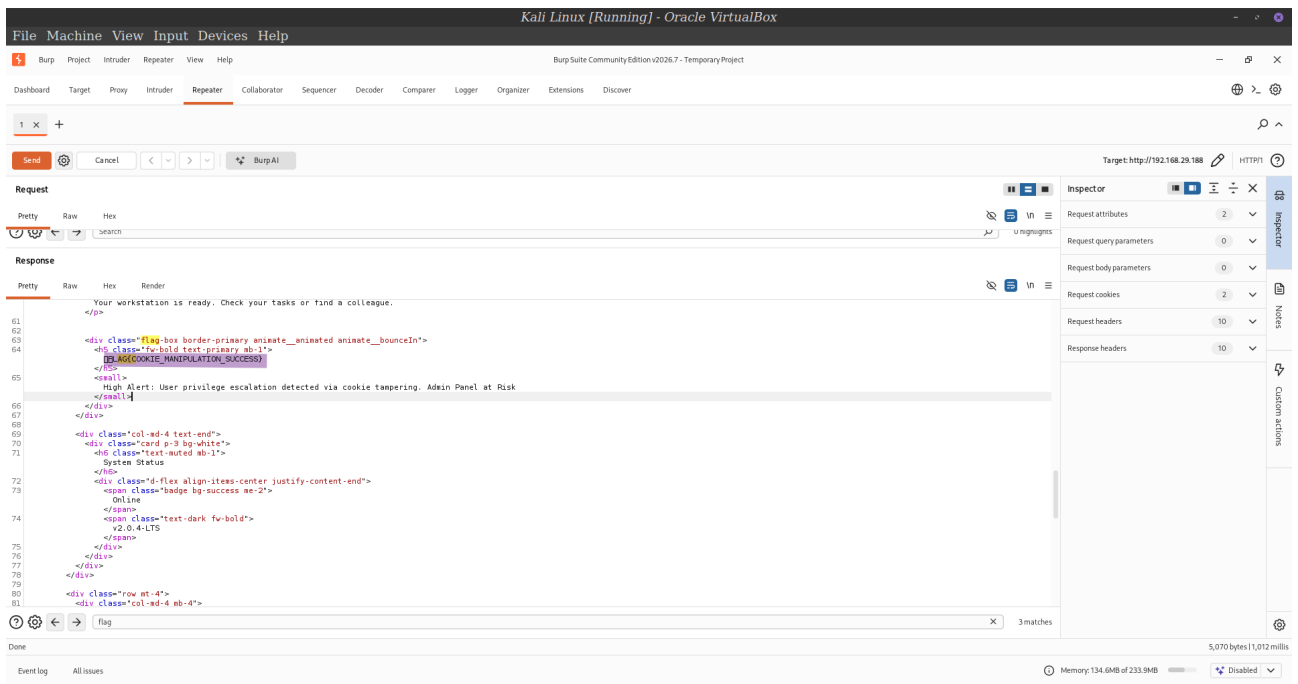
Step 6- Locating the Flag

Once Administrative access was available , I checked the page source for the flag.

Searching for the keyword flag then revealed the assignment flag.

Step 7- Screenshot Evidence





Answer the questions below

Exploit an **improper session management** vulnerability on the site and capture the flag

FLAG(COOKIE_MANIPULATION_SUCCESS)

✓ Correct Answer



```
</head>
<body>

<nav class="navbar navbar-expand-lg navbar-dark sticky-top mb-4">
  <div class="container">
    <a class="navbar-brand fw-bold" href="index.php">
      <span class="text-primary">Secure</span>Corp
    </a>
  </div>
</nav>

<div class="container">
  <div class="row justify-content-center align-items-center animate__animated animate__fadeInDown" style="min-height: 70vh;">
    <div class="col-md-4">
      <div class="text-center mb-4">
        <h2 class="fw-bold"><span class="text-primary">Secure</span>Corp</h2>
        <p class="text-muted">Internal Employee Access</p>
      </div>
      <div class="card p-4 shadow border-0">
        <form method="POST">
          <div class="mb-3">
            <label class="form-label fw-bold small">Username</label>
            <input type="text" name="username" class="form-control" placeholder="Enter username" required>
          </div>
          <div class="mb-3">
            <label class="form-label fw-bold small">Password</label>
            <input type="password" name="password" class="form-control" placeholder="*****" required>
          </div>
          <button type="submit" class="btn btn-primary w-100 shadow-sm mb-3">Sign In</button>
          <div class="text-center">
            <a href="forgot_password.php" class="text-decoration-none small text-muted">Forgot password?</a>
          </div>
        </form>
      </div>
      <div class="mt-4 text-center">
        <p class="x-small text-muted">© 2026 SecureCorp Infrastructure Team | <span class="badge bg-light text-dark">v1.0.4-beta</span></p>
        <!-- Hey John Welcome onboard use your first name as username and 'babayaga' as your password and remove this comment after you login -->
      </div>
    </div>
  </div>
</div>

<div>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</div>
<div>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</div>
<div>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</div>
</div>
</body>
</html>
```

Security Risks Identified

1- Exposure of Credentials

Hard-coded credentials in client-side source code can be discovered by anyone inspecting the page.

Risk: Unauthorized access to the application.

2- Client-Side Authorization

The application trusted client-controlled data when determining user privileges.

Risk: Users may gain access to functionality beyond their intended permissions

3- Improper Session Management

Session information should be managed securely on the server. If authorization depends on data that the client can modify, the integrity of the application's access control model is compromised.

Real-World Relevance

Improper session management is among the most significant web application security issues and is consistently highlighted in the **OWASP Top 10**.

Security professionals routinely assess session handling because flaws can result in:

- Unauthorized access to sensitive information
- Administrative account compromise
- Data breaches
- Unauthorized modification of application data
- Loss of customer trust
- Regulatory and compliance issues

Understanding how sessions and authorization work enables security analysts to identify weaknesses before they can be exploited in production environments.

Conclusion

This assignment provided practical experience in analyzing web application session management and understanding the importance of secure authorization mechanisms. By examining the application's source code, inspecting HTTP traffic with Burp Suite, and evaluating how session information was handled, it was possible to identify a weakness in the application's session management design. The assignment proved the importance of proper session management as a critical component of web application security and highlighted how minor implementation flaws can lead to significant security risks.